

RAG Files: What Each Script Covers

A file-by-file breakdown of the 9 uploaded files, mapped to RAG pipeline stages, plus what still needs to be learned to call this a complete RAG system.

1. Quick File Map

File	Role
company_policy.pdf	Source doc — clean, non-OCR (real text layer)
company_policy_OCR.pdf	Source doc — scanned/image version, needs OCR
pdf_checker.py	Diagnostic — detects OCR vs non-OCR PDF
create_db_with_pdf.py	Ingestion — non-OCR PDF → FAISS
create_db_with_OCR_pdf.py	Ingestion — scanned PDF → image → OCR → FAISS
create_db_with_hardcoded.py	Ingestion + Retrieval demo using hardcoded text (no PDF, no LLM)
reading_faiss.py	Inspection — opens a saved FAISS index, prints chunks + raw vectors
rag_chat_llm.py	Full RAG — ingestion + retrieval + prompt + local LLM answer, chat loop
db_comparison.png	Reference — vector DB options compared (FAISS, Chroma, Qdrant, Pinecone, pgvector)

2. Source Documents

company_policy.pdf (non-OCR)

A digitally-created PDF with a real, selectable text layer. Extracted text is 100% clean (one artifact: “persons.7.” run together — a page-break spacing issue in the source, not an OCR error).

company_policy_OCR.pdf (scanned)

A scanned/image version of the same policy. Confirms it needed OCR: the text “5 days” was misread as “\$ days” — a classic OCR character-recognition error (5 → \$), which would never happen with direct text extraction. This is the practical proof of why OCR quality depends on image clarity and font.

3. Diagnostic: pdf_checker.py

pdf_checker.py

Uses PyMuPDF to loop through each page and print the extractable text length and embedded image count. A text-based PDF shows high text length / 0 images; a scanned PDF shows ~0 text length / 1 full-page image. This is the correct first step before choosing which ingestion script (OCR or non-OCR) to run on a new document.

4. Ingestion Scripts

create_db_with_pdf.py (non-OCR ingestion)

Load (PyPDFLoader) → Split → Embed → Store → Save. Covers document loading, chunking, embeddings, and FAISS persistence for a clean text PDF.

■ **Bug to flag for students:** this script sets **chunk_size=50** and **chunk_overlap=50** — overlap equal to chunk size. Overlap must always be smaller than chunk_size; when they are equal (or overlap is larger), the splitter cannot make forward progress correctly and produces excessive, near-duplicate micro-chunks. Good teaching moment: show this next to `create_db_with_OCR_pdf.py` (size=500, overlap=50) to contrast a healthy ratio.

create_db_with_OCR_pdf.py (OCR ingestion)

PDF page → rendered image (PyMuPDF, 300 DPI) → OCR text (pytesseract) → wrapped into Document objects → same Split/Embed/Store/Save flow as the non-OCR script. Also saves each rendered page as a PNG for visual debugging of OCR quality.

create_db_with_hardcoded.py (concept demo, no PDF)

Skips PDF loading entirely — uses 3 hardcoded Document objects instead. Useful teaching tool because it isolates chunking + embedding + FAISS + retrieval from the messiness of PDF parsing. Also introduces the load-or-create pattern (check if faiss_index/ already exists) and demonstrates retrieval (as_retriever, k=2) end-to-end — but stops at showing retrieved chunks; it does **not** call an LLM to generate a final answer.

5. Inspection: reading_faiss.py

reading_faiss.py

Loads a saved FAISS index back from disk and inspects it two ways: (1) prints stored chunk text via **docstore._dict**, and (2) prints the raw numeric embedding for each vector via **index.reconstruct(i)**, alongside total vector count and dimension (384). This is a pure debugging/verification script — it doesn't do retrieval or generation, it just proves what actually got stored.

6. Full RAG: rag_chat_llm.py

rag_chat_llm.py

The only file that completes the full loop: load-or-build FAISS → retriever (k=3) → interactive chat loop → build context from retrieved chunks → construct a grounded prompt → send to a local LLM (distilgpt2 via HuggingFacePipeline) → print generated answer. This is a hand-written RetrievalQA implementation.

■ **Known limitation:** distilgpt2 is a small, non-instruction-tuned model (2019, ~82M params). It was not trained to follow instructions like “answer only from context,” so generated answers may ramble, repeat, or ignore the grounding instruction even though retrieval itself works correctly.

7. Reference: db_comparison.png

db_comparison.png

A comparison table (FAISS, Chroma, Qdrant, Pinecone, pgvector) across setup effort, metadata filtering, production scaling, and best-fit use case. Positions FAISS as best for learning/prototypes and shows what changes when moving toward a production deployment.

8. What This Set of Files Does NOT Cover

These 9 files together demonstrate a complete, working, minimal RAG pipeline (ingestion → retrieval → generation) for a single small PDF. To call this production-ready or fully understand RAG, the following are still missing:

- **Answer quality / instruction-tuned LLM** — Swap `distilgpt2` for an instruction-tuned model (e.g. Claude, GPT, Flan-T5, or a local Llama/Mistral via Ollama) so answers actually follow the grounding instruction instead of just predicting next tokens.
- **Evaluation** — No script measures whether answers are actually correct/grounded. Concepts to add: faithfulness, groundedness, context precision/recall, and frameworks like RAGAS.
- **Retrieval quality tuning** — Only plain similarity search with a fixed `k` is shown. Missing: similarity thresholds, MMR (diversity-aware retrieval), hybrid search (keyword + semantic), and re-ranking retrieved chunks with a second model.
- **Multi-document / larger scale** — All scripts use one small PDF (4 chunks). Real systems ingest many documents with per-document metadata (source, page, date) used for filtering and citation — not yet demonstrated here.
- **Conversational memory** — The chat loop in `rag_chat_llm.py` treats every question independently — it cannot handle follow-ups like “what about sick leave?” after a prior question. Needs chat history + query reformulation (ConversationalRetrievalChain equivalent).
- **Chunking strategy for structured documents** — `RecursiveCharacterTextSplitter` is generic. For policy-style documents with numbered sections, header-aware or semantic chunking (splitting by section rather than raw character count) usually retrieves cleaner, self-contained chunks.
- **Production vector DB** — FAISS here is a local file on disk, single-user, no concurrent write handling. Production systems typically move to Chroma, Qdrant, Pinecone, or pgvector (see `db_comparison.png`) for multi-user access, metadata filtering, and updates without full re-indexing.
- **Security & guardrails** — No handling of prompt injection (malicious text inside a PDF influencing the LLM), no access control over which documents a user's query can retrieve from, and no PII redaction before storing/embedding sensitive content.
- **Cost/latency in production LLMs** — `distilgpt2` runs free and local but is low quality. Real deployments need to reason about API cost, latency, and rate limits when swapping in a hosted LLM.
- **Agentic RAG** — Currently retrieval always happens exactly once per question with a fixed `k=3`. More advanced systems let the LLM decide whether to retrieve at all, reformulate the query, or retrieve multiple times before answering.

9. Suggested Next Teaching Steps

- Fix the `chunk_overlap` bug in `create_db_with_pdf.py` and show students the difference in chunk output before/after.
- Swap `distilgpt2` for an instruction-tuned model and compare answer quality on the same questions.
- Add metadata (page number, section) to chunks and show citing sources in the final answer.
- Introduce a second, larger PDF to demonstrate chunk count scaling and retrieval behavior with more data.
- Try MMR retrieval (`search_type="mmr"`) vs plain similarity search on a query where multiple chunks are similar.
- Run the same question through both the OCR and non-OCR FAISS indexes and compare retrieval accuracy — a direct, hands-on way to show how OCR errors (like § instead of 5) can affect retrieval.